[Nam Le] Userspace Debugging Toolkit: GDB, strace & Valgrind

1. System Debugging & Basic GDB

1.1 Why need Systematic Debugging?

```
Program has problem
      │
      ├── Crash
      ├── Hang
      ├── Wrong Result
      ├── Memory corruption
      └── Resource/system-call failure
```

Insted of (printf-debugging):

```
printf("here 1\n");
printf("here 2\n");
printf("here 3\n");
```

We have to:

```
GDB       → what is my code doing?
strace    → what is my program asking the OS to do?
Valgrind  → Is my program misuse memory?
```

1.2 Compile with debug information

Example:

```
/* text_debug.c */
#include <stdio.h>

int main() {
    int a = 5;
    int b = 10;
    int unused_result = 0;

    printf("--- Start compute---\n");

    for (int i = 0; i < 3; i++) {
        unused_result += (a * b);
        a += 2;
    }

    int final_score = a + b;
    printf("Final result: %d\n", final_score);

    return 0;
}
```

Know that:
`gcc ./test_debug.c -o test_debug` **differ from** `gcc -g -O0 ./test_debug.c -o test_debug_0` and:
`gcc -g -O0 ./test_debug.c -o test_debug_0` **differ from** `gcc -g -O2 ./test_debug.c -o test_debug_2`

```
-g
 |
 └── add debugging information into executable
```

```
-O0
 |
 └── limit/restrict compiler optimization

 -O2
 |
 └── optimized mode
```

NOTE:

- Use -g on execute, you can debug the information in program
- Use -O0 instead of -O2
  - (-O0) use *next* (in gdb) to debug, jump each once line

```
    (gdb) run
Starting program: /home/hoainam0508/Documents/debugging/test_debug_0
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at test_debug.c:4
warning: Source file is more recent than executable.
4           int a = 5;
(gdb) n
5           int b = 10;
(gdb) n
6           int unused_result = 0;
(gdb) n
8           printf("--- Start compute---\n");
(gdb) n
--- Start compute---
10          for (int i = 0; i < 3; i++) {
```

  - (-O2) jump wildly (executaion sort the Assembly command for CPU can run faster

```
(gdb) break main
Breakpoint 1 at 0x1080: file test_debug.c, line 3.
(gdb) r
Starting program: /home/hoainam0508/Documents/debugging/test_debug_2
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
```

```
Breakpoint 1, main () at test_debug.c:3
warning: Source file is more recent than executable.
3       int main() {
(gdb) n
112       return __printf_chk (__USE_FORTIFY_LEVEL - 1, __fmt, __va_arg_pack
());
(gdb) n
8           printf("--- Start compute---\n");
(gdb) n
--- Start compute---
16          printf("Final result: %d\n", final_score);
(gdb) n
Final result: 21
18          return 0;
```

1.3 Basic GDB (Lab 1)

Start GDB: `gdb ./program` Basic GDB command :

```
run(r): start (or restart again) program
continue(c):  Let program can continue run normally, with breakpoint, program will continue to
run until:
    - meet next BreakPoint
    - crash (by program)
    - terminated completely (by program)
quit(q) / exit: out gdb environment
```

BreakPoint: (b)
establishment

- break main: place break point at main function
- break function_name: place break point at specified function
- break file.c:20 : place break pointer at specified line in program
- break (**no argument**): set **breakpoint** in next line (of current stack frame)
- break with condition: `break ... if cond`

inspect BreakPoint list (**info break - i b**)

```
(gdb) r
Starting program: /home/hoainam0508/Documents/debugging/test_debug_0
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at test_debug.c:4
warning: Source file is more recent than executable.
4           int a = 5;
(gdb) i b
Num     Type           Disp Enb Address            What
1       breakpoint     keep y   0x0000555555555175 in main at test_debug.c:4
```

- num: id for break point
- type: type of point (break, watch, catchpoint)
- Disp: state after break (**keep** or **del**)
- Enb: State is enable (y) or (n) disabled
- Address:
- What: position in program

Delete/Clear BreakPoint:

- Delete [list] : delete break point for each id . Ex: delete 1; delete 1,2
- Clear: function_name / file.c:20

Stepping:

- step (s): execute next line, If has function call, **step** jump to this function call, then execute
- next (n): execute next line, If has function call, **next** will skip this function, no jump into function call
  - can use step [count] or next[count] to jump far
- finish (fin): finish current function, then return the start of this function --> return value of function (if have) **NOTE**: cant use for **main** function

WatchPoint: notify when has change with variable
Example: (when change x, x++)

```
14              x++;
(gdb) n

Hardware watchpoint 3: x

Old value = 1
```

```
New value = 2
main () at ./gdb_test.c:15
```

BACKTRACE (bt): display summary the list of function call, then we know the functions are arrived
Example:

```
#include <stdio.h>

void foo() {
    int *p = NULL;
    printf("Before crash...\n");
    *p = 42;   // SEGFAULT  here
}

void bar() {
    printf("Inside bar()\n");
    foo();
}

void process() {
    printf("Inside process()\n");
    bar();
}

int main() {
    printf("Inside main()\n");
    process();
    return 0;
}
```

Result:

```
Program received signal SIGSEGV, Segmentation fault.
0x0000555555555170 in foo () at ./backtrace.c:6
6           *p = 42;   // SEGFAULT  here
(gdb) bt
#0  0x0000555555555170 in foo () at ./backtrace.c:6            /*  current frame */
#1  0x000055555555519a in bar () at ./backtrace.c:11
#2  0x00005555555551be in process () at ./backtrace.c:16
#3  0x00005555555551e2 in main () at ./backtrace.c:21
```

Core Dump
What is crash?
Program is terminated immediately by OS because violate a certain rule which CPU/OS cannot continue execute. **In nature**, CPU receives a hardware **exception/fault**, **kernel** send a **signal** to kill process

What is **core dump**?

```
Program is running
        ↓
       crash
        ↓
OS creates core dump
        ↓
core file contain state of process in the crash moment
        ↓
GDB read core
```

After apprear **crash** fault in above example, we generate a core file: `(gdb) generate-core-file`
Result:

```
warning: Memory read failed for corefile section, 4096 bytes at 0xfffffffff600000.
Saved corefile core.101983
```

So we have core file *core.101983*
Then run **(both 2 files)** `gdb ./backtrace ./core.101983` to check SEGMEN FAULT (in summary)

```
Core was generated by `/home/hoainam0508/Documents/debugging/backtrace'.
Program terminated with signal SIGSEGV, Segmentation fault.
#0  0x0000555555555170 in foo () at ./backtrace.c:6
6□    *p = 42;   // SEGFAULT  here
```

1.4 GDB Multi-Thread

Example:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
```

```
void foo() {
    int *p = NULL;
    printf("Thread crashing...\n");
    *p = 42;   // SEGFAULT here, not in main thread
}

void* worker(void* arg) {
    printf("Inside worker thread\n");
    sleep(1);      // ensure run after main thread
    foo();
    return NULL;
}

int main() {
    pthread_t t;
    printf("Inside main()\n");
    pthread_create(&t, NULL, worker, NULL);
    pthread_join(t, NULL);
    return 0;
}
```

Compile: `gcc -g thread_gdb.c -o thread_gdb -pthread`
Result (in gdb):

```
(gdb) r
Starting program: /home/hoainam0508/Documents/debugging/thread_gdb
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Inside main()
[New Thread 0x7ffff7bff640 (LWP 105849)]
Inside worker thread
Thread crashing...

Thread 2 "thread_gdb" received signal SIGSEGV, Segmentation fault.
[Switching to Thread 0x7ffff7bff640 (LWP 105849)]
0x0000555555551f0 in foo () at thread_gdb.c:8
8           *p = 42;   // SEGFAULT here, not in main thread
(gdb) info thread                                                    /* Check info
threads */
  Id   Target Id                                    Frame
```

```
  1    Thread 0x7ffff7fa8740 (LWP 105846) "thread_gdb" __futex_abstimed_wait_common64
(private=128, cancel=true, abstime=0x0, op=265, expected=105849, futex_word=0x7ffff7bff910)
    at ./nptl/futex-internal.c:57
* 2    Thread 0x7ffff7bff640 (LWP 105849) "thread_gdb" 0x00005555555551f0 in foo () at
thread_gdb.c:8
```

Some command when use gdb with threads:

Command	Functional
info threads	list all thread, active = `*`
thread N	change focus GDB to thread N
thread apply N backtrace	watch *backtrace* of thread N (no switch)
thread apply all backtrace	watch *backtrace* of all thread
break foo thread 2	breakpoint = Thread 2

Want to see backtrace of all thread (example above)

```
(gdb) thread apply all bt

Thread 2 (Thread 0x7ffff7bff640 (LWP 105849) "thread_gdb"):
#0  0x00005555555551f0 in foo () at thread_gdb.c:8
#1  0x000055555555522c in worker (arg=0x0) at thread_gdb.c:14
#2  0x00007ffff7c94a83 in start_thread (arg=<optimized out>) at ./nptl/pthread_create.c:442
#3  0x00007ffff7d26890 in clone3 () at ../sysdeps/unix/sysv/linux/x86_64/clone3.S:81

Thread 1 (Thread 0x7ffff7fa8740 (LWP 105846) "thread_gdb"):
#0  __futex_abstimed_wait_common64 (private=128, cancel=true, abstime=0x0, op=265,
expected=105849, futex_word=0x7ffff7bff910) at ./nptl/futex-internal.c:57
#1  __futex_abstimed_wait_common (cancel=true, private=128, abstime=0x0, clockid=0,
expected=105849, futex_word=0x7ffff7bff910) at ./nptl/futex-internal.c:87
#2  __GI___futex_abstimed_wait_cancelable64 (futex_word=futex_word@entry=0x7ffff7bff910,
expected=105849, clockid=clockid@entry=0, abstime=abstime@entry=0x0,
private=private@entry=128) at ./nptl/futex-internal.c:139
#3  0x00007ffff7c965e4 in __pthread_clockjoin_ex (threadid=140737349940800, thread_return=0x0,
clockid=0, abstime=0x0, block=<optimized out>) at ./nptl/pthread_join_common.c:105
```

--> **USEFUL WHEN Debug DEADLOCK and RACE CONDITION**

Debug multi-threaded crash (Lab 2)

Example:

```
/* race_condition.c */
int* counter ;
void* writer_thread(void* arg) {
    int thread_id = *((int*) arg);
    counter = malloc(sizeof(int));
    *counter = 0;
    printf("[Thread %d] is running... (tid = %lu)\n", thread_id, pthread_self());
    for (int i = 0; i < 1000000; i++) {
        (*counter)++;
    }
    printf("[Thread %d] completely!\n", thread_id);
    pthread_exit(NULL);
}
void* free_thread(void* arg) {
    usleep(100);     /* 0.1 ms */
    printf("[Free] Free counter now...");
    free(counter);
    printf("[Free] Free done!");
    pthread_exit(NULL);
}
int main() {
    pthread_t threads[2];
    int thread_ids[2] = {1, 2};

    printf("[Main] Start create 2 threads\n");

    pthread_create(&threads[0], NULL, writer_thread, (void*)&thread_ids[0]);
    pthread_create(&threads[1], NULL, free_thread, (void*)&thread_ids[1]);

    /*Join 2 threads together, wait for sucess of all 2 threads*/
    pthread_join(threads[0], NULL);
    pthread_join(threads[1], NULL);
    return 0;
}
```

We can see that while **writer** are writing on *counter*, **free** immediately free this pointer variable --> **writer** thread still fix this variable --> segmentation fault because access invalid memory
Use gdb to fix it:

```
(gdb) break main
(gdb) run
(gdb) watch counter               /* (observe address of counter) */
(gdb) watch *counter              /* observe value of counter */
.....................................................................
(gdb) n                          /* go to *counter = 0 of thread 2 */
[Switching to Thread 0x7ffff7bff640 (LWP 18715)]

Thread 2 "race_condition" hit Hardware watchpoint 4: *counter

Old value = <unreadable>
New value = 0                               /* set value of counter */

Thread 2 "race_condition" hit Hardware watchpoint 5: counter

Old value = (int *) 0x0
New value = (int *) 0x7ffff0000b70            /* address of counter */
writer_thread (arg=0x7fffffffd498) at race_condition.c:9
9           *counter = 0;
(gdb) s
[New Thread 0x7ffff73fe640 (LWP 18755)]                     /* jump to printf function
(normally) */
10          printf("[Thread %d] is running... (tid = %lu)\n", thread_id, pthread_self());

(gdb) s                                                      /* use s, want to jump to for loop
*/
[Switching to Thread 0x7ffff73fe640 (LWP 18755)]         /* but Jump Thread here */

Thread 3 "race_condition" hit Hardware watchpoint 4: *counter

Old value = 0
New value = -65536                                       /* that mean free(counter) is active
*/
tcache_put (tc_idx=0, chunk=0x7ffff0000b60) at ./malloc/malloc.c:3184
3184    ./malloc/malloc.c: No such file or directory.
(gdb) s
[Free] Free counter now...[Thread 1] is running... (tid = 140737349940800)    /* return thread
2 Writer */
3185    in ./malloc/malloc.c                                                 /* That mean OS
```

```
skip for loop */

(gdb) thread 2                    /* trying to jump to thread 2  again */
[Switching to thread 2 (Thread 0x7ffff7bff640 (LWP 18715))]
#0  writer_thread (arg=0x7fffffffd498) at race_condition.c:11
11          for (int i = 0; i < 1000000; i++) {
(gdb) s
Thread 2 "race_condition" hit Hardware watchpoint 4: *counter

Old value = -65536
New value = -65535                 /* value of counter is garbage now */
writer_thread (arg=0x7fffffffd498) at race_condition.c:11
11          for (int i = 0; i < 1000000; i++) {
```

2. Strace

Lab 3

Example:

```
int main() {
    printf("[1] Reading config file...\n");

    /* File not exist */
    int config_fd = open("/etc/myapp/config.conf", O_RDONLY);
    // printf("fd value = %d\n", config_fd);
    char buffer[128] = {0};
    read(config_fd, buffer, sizeof(buffer));   // fd = -1 but still call read(), no check
    printf("[1] Config loaded (buffer may be garbage/empty, no error shown)\n");

    // --- HANG: acquire file lock, always block if another process hold lock---
    printf("[2] Acquiring lock file...\n");
    int lock_fd = open("/tmp/myapp.lock", O_CREAT | O_RDWR, 0644);
    if (lock_fd < 0) {
        perror("open lock file");
        return 1;
    }
```

```
    flock(lock_fd, LOCK_EX);   // BLOCKING call — hang if lock is held
    printf("[3] Lock acquired! Doing work...\n");

    sleep(30);   // hold lock 30s to test lock is hang

    flock(lock_fd, LOCK_UN);
    close(lock_fd);
    printf("[4] Done, lock released.\n");

    return 0;
}
```

Result:

```
[1] Reading config file...
[1] Config loaded (buffer may be garbage/empty, no error shown)
[2] Acquiring lock file...
[3] Lock acquired! Doing work...
[4] Done, lock released.
```

--> NO error message is printed, although file not exits. **DANGEROUS ERROR**: no crash, no log but wrong **behavior**\

When we use *strace*: `strace ./test_strace` (**check root cause**)

```
openat(AT_FDCWD, "/etc/myapp/config.conf", O_RDONLY) = -1 ENOENT (No such file or directory)
read(-1, 0x7ffe0b3b99a0, 128)           = -1 EBADF (Bad file descriptor)
```

If program stop at `sleep(30)`
but immediately run strace : `strace ./test_strace` (**check hang error**)
So the result is:

```
openat(AT_FDCWD, "/tmp/myapp.lock", O_RDWR|O_CREAT, 0644) = 3
flock(3, LOCK_EX
```

--> no return, hang forever here, **Strace** allow we can see where function is blocked!

3. Valgrind

Overview:
basic: `valgrind ./program`
detail check: `valgrind --leak-check=full \ --show-leak-kinds=all \ ./program`

Mem leak:

```
int *p = malloc(sizeof(int));
*p = 10;
/* forgot free */
```

valgrind: `definitely lost`

Invalid read/write:

```
int *p = malloc(3 * sizeof(int));
p[3] = 10;   /* OUT OF BOUNDS */
```

valgrind: `Invalid write`

Use-after-free:

```
int *p = malloc(sizeof(int));
free(p);
*p = 10;
```

Uninitialized memory:

```
int x;
printf("%d\n", x);
```

Valgrind: `Conditional jump or move depends on uninitialised value`

Revision #3
Created 2026-09-18 01:55:06 UTC by Lê Hoài Nam
Updated 2026-09-23 08:15:24 UTC by Lê Hoài Nam